

Jahnvi Chaudhary

Software Development Engineer · Backend Systems & Distributed Computing (Java)

Pune, Maharashtra | +91-8595754872 | chaudharyjahnvi89@gmail.com | github.com/jahnvichaudhary | [LinkedIn](#) | [Portfolio](#) | [LeetCode](#): Jahnvi_Chaudhary

Education

DES Pune University

2025 – 2027 (Pursuing)

Master of Computer Applications (Cloud & Distributed Systems)

Pune, MH

• CGPA: 7.53 (1st Year) · Coursework: Distributed Systems, Cloud Computing, Advanced DSA, DBMS, Computer Networks

JIMS, Greater Noida

2022 – 2025

Bachelor of Computer Applications

Greater Noida, UP

• First Division

Technical Skills

Languages & Fundamentals: Java 17, Python 3, SQL, Data Structures & Algorithms, OOP & SOLID Principles, Multithreading & Concurrency

Backend & Frameworks: Spring Boot 3, Spring MVC, Spring Data JPA, Spring Security, Hibernate, JWT, REST API Design, Apache Kafka, Maven

Data, Cache & Messaging: MySQL, PostgreSQL, Redis (Lua scripting), Apache Kafka, Base62 Encoding

Cloud & DevOps: Docker, Kubernetes, AWS (EC2, S3, RDS, EKS, IAM), GitHub Actions

Practices & Tools: System Design (LLD/HLD), Microservices Architecture, Agile/Scrum, Git, Postman

Backend Engineering Projects

Distributed Rate Limiter — shared-state rate limiting at scale

github: Distributed-Rate-Limiter

Java 17, Spring Boot, Redis, Lua, Docker Compose

- Designed a multi-instance rate limiter using two algorithms — sliding window log and token bucket — as atomic Lua scripts inside Redis, eliminating race conditions that occur when app-local counters (ConcurrentHashMap/Buffer4j) are used across scaled instances.
- Benchmarked 9,400 req/s on a single instance (p99: 61ms) and 14,000 req/s across two instances sharing one Redis node before Redis CPU became the bottleneck; verified shared-state correctness with concurrent dual-instance load tests.
- Made and documented a deliberate fail-open availability tradeoff on Redis outage, and authored a scale-out plan covering Redis Cluster sharding, hierarchical (user/tenant/global) limits, and edge-layer enforcement (Envoy/NGINX).

Real-Time Notification Service — event-driven push architecture

github: NotificationService-using-Kafka

Java 17, Spring Boot, Apache Kafka (KRaft), WebSocket, Docker

- Built an event-driven notification gateway that consumes Kafka topic events and pushes real-time updates to browser clients over WebSocket, replacing latency- and DB-load-heavy polling.
- Implemented a server-side topic-to-session subscription registry so clients receive only the events they subscribed to, avoiding the bandwidth and data-leakage cost of full-broadcast fan-out.
- Ran on KRaft-mode Kafka (no ZooKeeper); designed a horizontal scale-out path — Redis-backed shared subscription state, sticky-session routing, schema registry, and dead-letter topic handling — for multi-instance deployment.

Concurrent Task Queue (TaskRunner) — custom Java concurrency, no external broker

github: Concurrent-Task-Queue

Java 17, Spring Boot, MySQL, ExecutorService, BlockingQueue

- Engineered a background job-processing service with a hand-rolled worker pool (ExecutorService + BlockingQueue), supporting async execution, status polling, and exponential backoff retries (500ms → 4s) for failing jobs — deliberately avoiding Kafka/RabbitMQ where raw concurrency was the right-sized tool.
- Separated durable job state (MySQL) from in-memory execution ordering, enabling crash recovery by re-seeding queued/running jobs from the database on restart with zero work loss.
- Built an extensible handler-registry pattern (Spring auto-discovery via @Component) so new job types plug in without touching core dispatch logic.

URL Shortener — coordination-free ID generation at scale

Live | github: URLshortner

Java 21, Spring Boot 4, Redis, PostgreSQL, Docker, Snowflake, Base62

- Implemented Twitter's Snowflake algorithm for distributed, coordination-free ID generation (4M IDs/ms across 1024 nodes) and Base62 encoding to compress 64-bit IDs into 7-character codes — chosen over UUIDs specifically to avoid B-tree index fragmentation in PostgreSQL at scale.
- Applied a Redis Cache-Aside pattern on the redirect path, cutting database load by 95% on high-traffic links; used 302 redirects over 301 to preserve click tracking, and supported optional custom aliases with 409-on-conflict handling.
- Designed soft-delete semantics (isActive flag) to preserve analytics history on link deactivation, and documented rate-limiting and dynamic worker-ID leasing (etcd/ZooKeeper) as the next steps for safe multi-instance scaling.

Professional Experience

Backend Developer Intern

Jun 2024 – Jul 2024

Blazecian Technologies

Pune, MH

- Developed and shipped 3 RESTful microservices on AWS EC2 following enterprise OOP design principles, contributing to a production codebase alongside senior engineers.
- Configured Nginx reverse-proxy routing, Git-based CI/CD workflows, and multi-container Docker deployments for the service stack.
- Delivered features within Agile/Scrum sprints on Jira, with consistent participation in code reviews and test-module ownership.

Coding Profile & Certifications

- LeetCode** — 500+ problems solved (Java) across Arrays, Two Pointers, Dynamic Programming, Backtracking, Hash Tables, Recursion — leetcode.com/u/Jahnvi_Chaudhary
- AWS Certified** — Cloud Practitioner, DevOps, Networking, Monitoring & Security tracks